CT-159 · DATA STRUCTURES

Data Structures, Algorithms & Applications

Lecture 1 — Introduction to Data Structures & Abstract Data Types

Course Teacher: Dr. Muhammad Kamran

COURSE LOGISTICS

01

Course Requirements

Homework	
Assessments	10
Mini Project	5
Competition	5

- This may change

Examinations	
Midterm	20
Final Exam	60

BEFORE WE BEGIN

Phones on silent - No phones allowed during the lecture!

Teaching and learning in progress — let's keep the room focused.

SECTION 01

What is Data?

Data, terminology, and how information is organized into records and files.

SECTION 01 · DATA

02

What is a Data Structure?

> *A data structure is a systematic way to organize data so it can be used efficiently.*

- It is the group of data elements that provides an efficient way of storing and organizing data in the computer.
- The goal is always efficiency — in both time (speed of operations) and space (memory used).
- Choosing the right structure for a problem is one of the core skills of this course.

SECTION 01 · DATA

03

Defining Data

A good data definition has four characteristics:

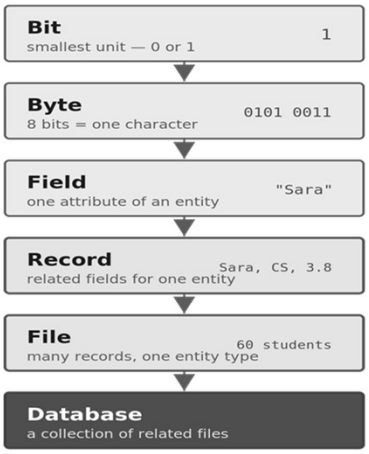
Atomic	Should define a single concept.
Traceable	Should be able to be mapped to some data element.
Accurate	Should be unambiguous.
Clear & Concise	Should be understandable.

SECTION 01 · DATA

04

Data Terminology — Hierarchy

Information is organized into a hierarchy: fields build records, records build files.



increasing size & complexity

Reading it in plain words

A database is a collection of files.

A file is made of records.

A record is made of fields.

A field is made of bytes (characters).

A byte is made of 8 bits.

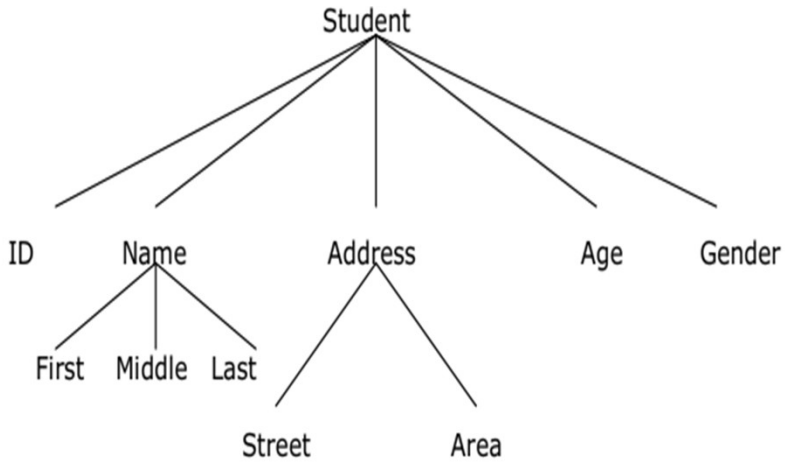
Each level is built from the level below it.

SECTION 01 · DATA

05

Data Terminology — Hierarchy

Information is organized into a hierarchy: fields build records, records build files.



SECTION 01 · DATA

06

Terminology — Building Blocks

Data	An elementary value or collection of values — e.g. a student's name and ID.
Entity & Attribute	An entity is a class of objects; each attribute is a property of that entity.
Field	A single elementary unit of information representing one attribute.

SECTION 01 · DATA

07

Terminology — Grouping Data

Group Items	Data items with subordinate items — e.g. a name splits into first + last.
Record	A collection of related data items grouped together (name, address, course, marks).
File	A collection of records of one entity type — 60 employees → 60 records.

SECTION 01 · DATA

08

Core Operations

Traversing

Visit every element once

Insertion

Add a new element

Deletion

Remove an element

Searching

Find a given element

Sorting

Arrange elements in order

Merging

Combine two structures

SECTION 02

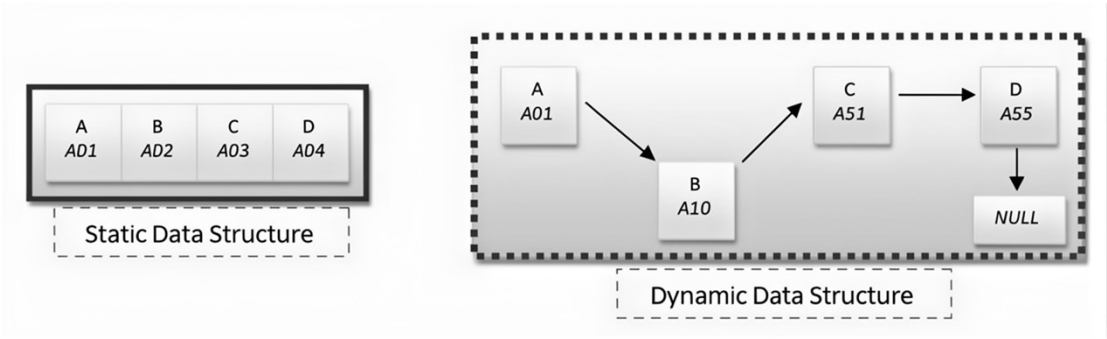
Types of Data Structures

From primitive types to arrays, lists, trees and graphs.

SECTION 02 · TYPES

09

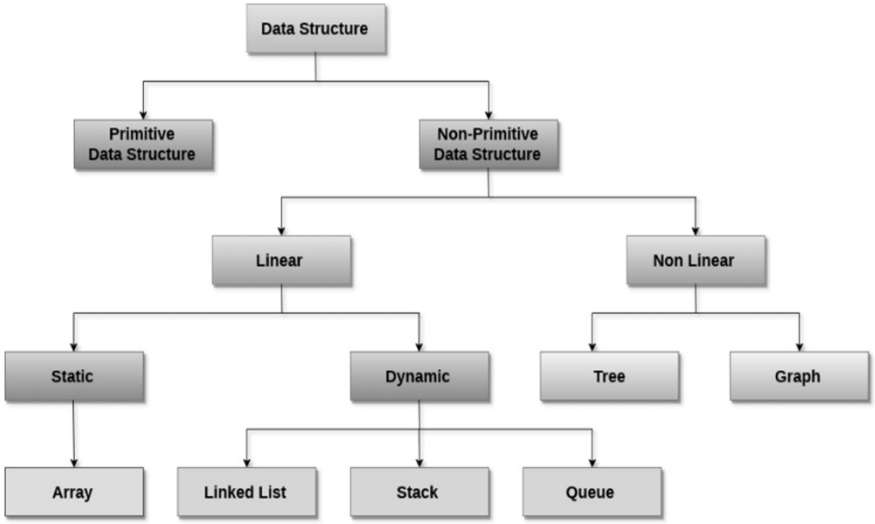
Classification of Data Structures



SECTION 02 · TYPES

10

Classification of Data Structures



SECTION 02 · TYPES

11

Primitive Data Structures

Hold a single value in a specific location. The four primitive types:

Integer	Whole numbers, positive or negative. Use long for larger ranges. (...-3,-2,-1,0,1,2,3,..)
Float	Decimal values. Use double when higher precision is needed. (1.1, 0.333333, ...)
Boolean	Holds True or False — mainly for checking conditions.
Character	A single character, e.g. 'A' or 'a'.

SECTION 02 · TYPES

12

Non-Primitive Data Structures

- Can hold multiple values, in either contiguous or random locations.
- Defined by the programmer (not built into the language as primitives).
- Classified further into linear and non-linear structures.

Examples

Array | Stack | Queue | Linked List | Tree | Graph

SECTION 02 · TYPES

13

Characteristics of a Data Structure

Every structure is described on three axes:

Correctness	The interface correctly implements the intended behaviour.
Time Complexity	How running time grows as input size grows.
Space Complexity	How much memory the structure and operations require.

Execution-time cases: Best · Worst · Average

SECTION 02 · TYPES

14

Why Do We Need Data Structures?

Processor speed	Huge data volumes need efficient organisation to process at speed.
Data search	Scanning 1006 items one-by-one every query slows the whole system.
Multiple requests	Thousands of simultaneous users can overwhelm an unoptimised server.

SECTION 03

Algorithms

What an algorithm is, why we need them, and how we measure their cost.

What is an Algorithm?

A finite set of instructions, carried out in a specific order, to perform a task.

- A process or set of rules for calculations and problem-solving, especially by a computer.
- Must terminate after a finite number of well-defined steps.

SECTION 03 · ALGORITHMS

16

Why Do We Need Algorithms?

Scalability	Break a large real-world problem into small, analysable steps.
Performance	A problem that decomposes cleanly into steps is a feasible one to solve.

SECTION 03 · ALGORITHMS

17

Example — Sum of Two Numbers

1. Start
2. Declare three variables a, b, and sum
3. Enter the values of a and b
4. Add: sum = a + b
5. Print sum
6. Stop

SECTION 03 · ALGORITHMS

18

Characteristics of an Algorithm (1/2)

Input	Takes zero or more input values.
Output	Produces one or more outputs at the end.
Unambiguity	Instructions are clear and simple — no room for interpretation.

SECTION 03 · ALGORITHMS

19

Characteristics of an Algorithm (2/2)

Finiteness	Contains a limited, countable number of instructions.
Effectiveness	Every instruction meaningfully affects the overall process.
Language independent	Can be implemented in any language with the same result.

SECTION 04

Algorithm Complexity

Measuring the cost of an algorithm in time and space.

Time Complexity

- The amount of time required to complete execution.
- Denoted using Big-O notation — the asymptotic notation for growth rate.
- Calculated by counting the number of steps to finish execution.

Big-O describes how running time scales as input size n grows.

SECTION 04 · COMPLEXITY

21

Space Complexity

- The amount of memory required to solve a problem and produce output.
- Like time complexity, it is expressed in Big-O notation.

Three components

Instruction space	Data space	Environment space
code stored in memory	variables & constants	call stack & runtime

SECTION 04 · COMPLEXITY

22

Program = Algorithm + Data Structure



Learning data structures and algorithms lets us build efficient, optimized programs.

SECTION 04 · COMPLEXITY

A Computer Program (C++)

```
#include <iostream>
using namespace std;

int main() {
    int first_number, second_number, sum;

    cout << "Enter two integers: ";
    cin  >> first_number >> second_number;

    // add the two numbers
    sum = first_number + second_number;

    cout << first_number << " + " << second_number
         << " = " << sum;
    return 0;
}
```

SECTION 05

Abstract Data Types

Separating what a structure does from how it is built.

SECTION 05 · ADTS

24

Abstract Data Types (ADT)

An ADT is a model of a data structure that specifies what it does — not how it is implemented.

- Specifies the characteristics of the data collection and the operations allowed on it.
- Abstract because it hides low-level implementation details.
- A single ADT can have many different implementations.

SECTION 05 · ADTS

25

Example — A Bag

- A bag is a container for a group of data items (analogy: a bag of candy).
- Position doesn't matter: {3, 2, 10, 6} is the same bag as {2, 3, 6, 10} — unlike a list.
- Items need not be unique: {7, 2, 10, 7, 5} isn't a set, but it is a valid bag.

SECTION 05 · ADTS

26

The Bag Interface

add(item)	Add an item to the bag.
remove(item)	Remove one occurrence of item.
contains(item)	Check whether item is in the bag.
numItems()	Get the number of items.
grab()	Get a random item without removing it.
toArray()	Return an array of the bag's contents.

SECTION 05 · ADTS

27

ADTs Are All Around You

You already use abstract data types every day — you press buttons (the interface) and never see the machinery (the implementation).

Vending Machine

`select(item) · pay() · collect()`

You press a button. How it's stocked or wired inside is hidden — and irrelevant to you.

TV Remote

`power() · volumeUp() · channel(n)`

Everyone knows what the buttons do. Almost nobody knows how the TV carries it out.

ATM

`withdraw() · deposit() · balance()`

A fixed set of operations. The bank can rebuild the machinery behind it — your buttons stay the same.

SECTION 05 · ADTS

28

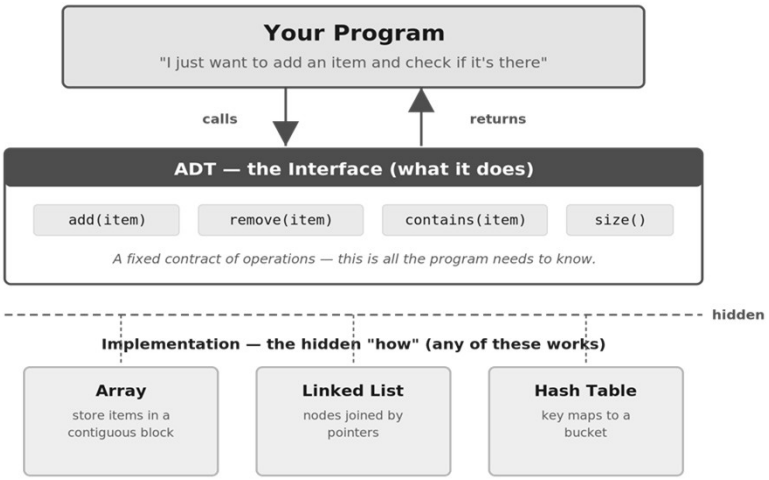
The One Big Idea

WHAT it does	HOW it does it
• **The interface** • The list of operations you can call • Visible to the user of the ADT • **Stays the same**	• **The implementation** • The actual code and storage • Hidden from the user • **Can change freely**

SECTION 05 · ADTS

29

The Abstraction Process

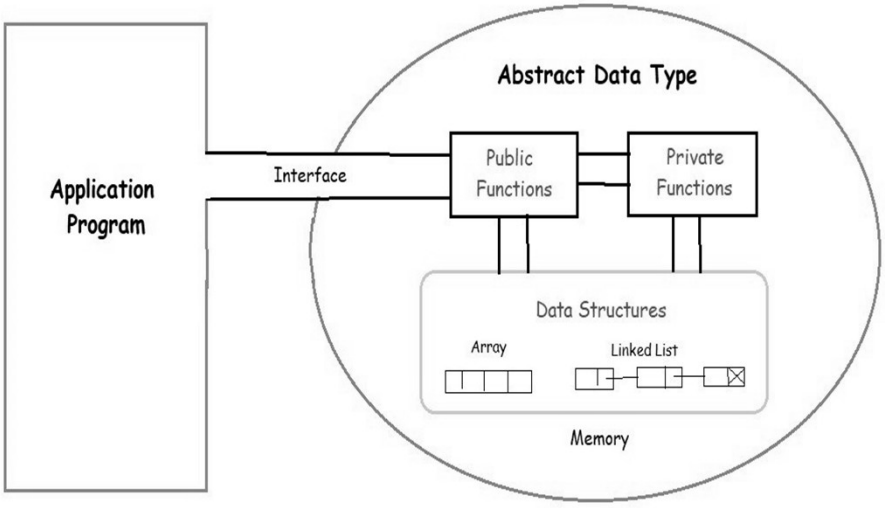


Swap the implementation and the program above never changes — that is abstraction.

SECTION 05 · ADTS

30

Abstraction Process



SECTION 05 · ADTS

31

Features of an ADT

- Abstraction
- Better conceptualization
- Robustness
- Encapsulation
- Data-structure independence
- Information hiding
- Modularity

LECTURE 1 — RECAP

What we covered

- Data, terminology, records & files
- Primitive vs non-primitive types
- Algorithms and their characteristics
- Time & space complexity (Big-O)
- Abstract Data Types & the Bag example

Next: Arrays, Linear Search & Binary Search